

Matplotlib (1)

2026-1학기

ICT융합학부 컴퓨터공학트랙

김대환

목차

1. 일반적인 Matplotlib 사용법
2. 간단한 라인 플롯
3. 간단한 산점도
4. 밀도 플롯과 등고선 플롯
5. 플롯 범례 맞춤 변경하기
6. 색상 막대 맞춤 변경하기
7. 다중 서브플롯

일반적인 Matplotlib 사용법

일반적인 Matplotlib 사용법 (1)

■ Matplotlib импорт

- Matplotlib импорт하기

```
import matplotlib as mpl
import matplotlib.pyplot as plt
```

- 스타일 설정하기

```
plt.style.use('classic')
```

- plt.style 지시어 사용

- Show() 사용 여부 – 플롯 표현 방법

- 스크립트에서 플로팅

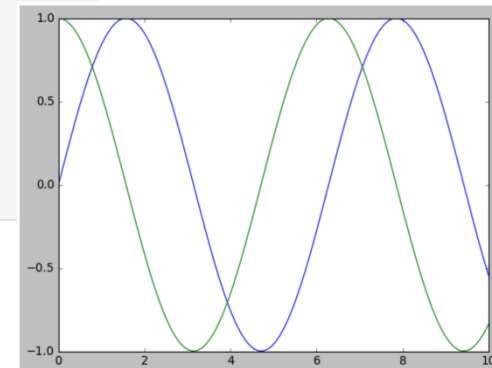
- **plt.show()**: 이벤트 루프를 시작해 현재 활성화된 모든 그림 객체를 찾아서 그림을 표시 (하나 이상의 대화장을 오픈)

```
# ----- file: myplot.py -----
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 100)

plt.plot(x, np.sin(x))
plt.plot(x, np.cos(x))

plt.show()
```



일반적인 Matplotlib 사용법 (2)

■ Matplotlib импорт (2)

■ Show() 사용 여부 – 플롯 표현 방법 (2)

■ Ipython 셀에서 플로팅

- ipython을 시작한 후 **%matplotlib** 매직 명령어를 사용

```
In [1]: %matplotlib
Using matplotlib backend: TkAgg
```

```
In [2]: import matplotlib.pyplot as plt
```

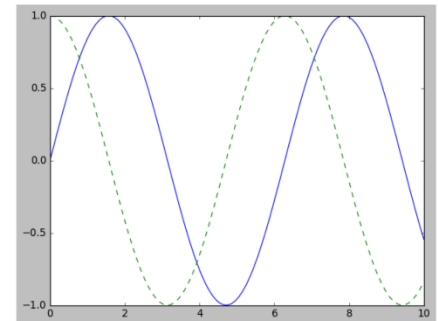
■ Jupyter 노트북에서 플로팅

- **%matplotlib notebook** : 노트북 내에서 대화형 플롯 사용
- **%matplotlib inline** : 노트북에 플롯의 정적 이미지를 삽입

기능	%matplotlib inline	%matplotlib notebook
출력 형태	정적 이미지	인터랙티브 그래프
확대/이동	✗ 불가능	✓ 가능 (줌, 드래그)
업데이트	✗ 안 됨	✓ 실시간 반영
속도/안정성	✓ 빠르고 안정적	✗ 느리고 가끔 버그
논문/보고서	✓ 적합	✗ 부적합

```
import numpy as np
x = np.linspace(0, 10, 100)

fig = plt.figure()
plt.plot(x, np.sin(x), '-')
plt.plot(x, np.cos(x), '-');
plt.show()
```



일반적인 Matplotlib 사용법 (3)

■ Matplotlib 임포트 (3)

■ 그림을 파일로 저장하기

- `savefig()` `fig.savefig('my_figure.png')`

- 그림을 저장
- 파일 형식은 지정한 파일명의 확장자로 추론

```
fig.canvas.get_supported_filetypes()
```

```
{'eps': 'Encapsulated Postscript',  
 'jpg': 'Joint Photographic Experts Group',  
 'jpeg': 'Joint Photographic Experts Group',  
 'pdf': 'Portable Document Format',  
 'pgf': 'PGF code for LaTeX',  
 'png': 'Portable Network Graphics',  
 'ps': 'Postscript',  
 'raw': 'Raw RGBA bitmap',  
 'rgba': 'Raw RGBA bitmap',  
 'svg': 'Scalable Vector Graphics',  
 'svgz': 'Scalable Vector Graphics',  
 'tif': 'Tagged Image File Format',  
 'tiff': 'Tagged Image File Format',  
 'webp': 'WebP Image Format'}
```

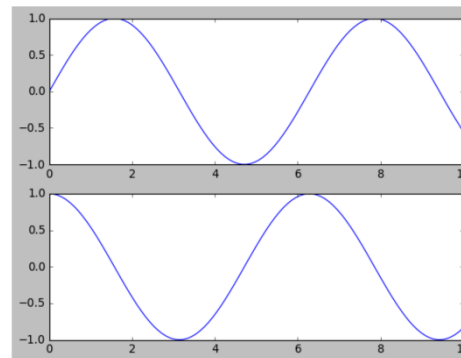
일반적인 Matplotlib 사용법 (4)

- Matplotlib 인터페이스 (1)
 - 종류
 - 매트랩 스타일의 상태 기반 인터페이스
 - 객체지향 인터페이스
 - 매트랩 스타일의 상태 기반 인터페이스
 - 원래 매트랩 (Matlab) 사용자를 위한 파이썬 대안으로 제작
 - 파이플롯 (pyplot, plt) 인터페이스에 포함
 - 상태를 저장한다는 개념
 - `plt.gcf()`: 현재 그림을 읽음, `plt.gca()`: 현재 축을 읽음

```
plt.figure() # create a plot figure

# create the first of two panels and set current axis
plt.subplot(2, 1, 1) # (rows, columns, panel number)
plt.plot(x, np.sin(x))

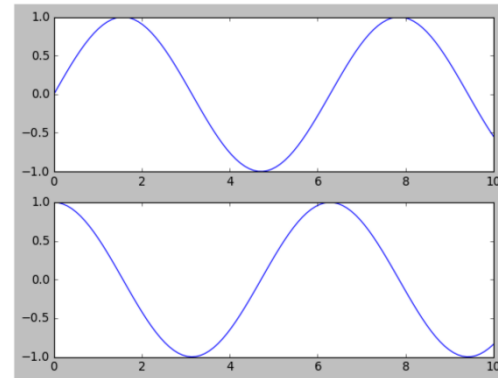
# create the second panel and set current axis
plt.subplot(2, 1, 2)
plt.plot(x, np.cos(x));
plt.show()
```



일반적인 Matplotlib 사용법 (5)

- Matplotlib 인터페이스 (2)
 - 객체 지향 인터페이스
 - 활성화된 그림이나 축 개념에 의존하지 않음
 - 명시적인 Figure와 Axes 객체 메소드

```
# First create a grid of plots  
# ax will be an array of two Axes objects  
fig, ax = plt.subplots(2)  
  
# Call plot() method on the appropriate object  
ax[0].plot(x, np.sin(x))  
ax[1].plot(x, np.cos(x));  
plt.show()
```



간단한 라인 플롯

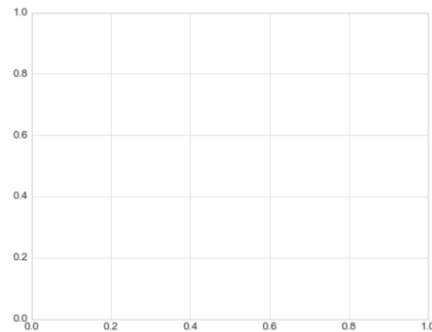
간단한 라인 플롯 (1)

■ 라인 플롯

- 모든 Matplotlib 플롯은 그림 (figure)와 축 (axes)를 만드는 것으로 시작

```
%matplotlib inline
import matplotlib.pyplot as plt
plt.style.use('seaborn-v0_8-whitegrid')
import numpy as np
```

```
fig = plt.figure()
ax = plt.axes()
plt.show()
```



```
import matplotlib.pyplot as plt
print(plt.style.available)
```

```
['Solarize_Light2', '_classic_test_patch', '_mpl-gallery', '_mpl-gallery-nogrid', 'bmh',
'classic', 'dark_background', 'fast', 'fivethirtyeight', 'ggplot', 'grayscale', 'seaborn-v0_8',
'seaborn-v0_8-bright', 'seaborn-v0_8-colorblind', 'seaborn-v0_8-dark', 'seaborn-v0_8-dark-palette',
'seaborn-v0_8-darkgrid', 'seaborn-v0_8-deep', 'seaborn-v0_8-muted', 'seaborn-v0_8-notebook',
'seaborn-v0_8-paper', 'seaborn-v0_8-pastel', 'seaborn-v0_8-poster', 'seaborn-v0_8-talk',
'seaborn-v0_8-ticks', 'seaborn-v0_8-white', 'seaborn-v0_8-whitegrid', 'tableau-colorblind10']
```

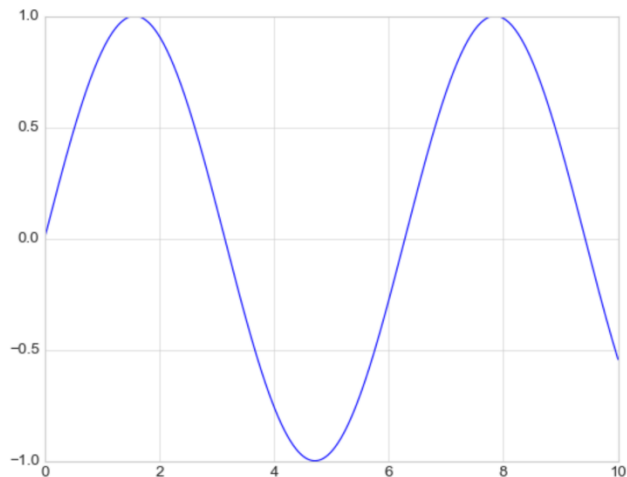
카테고리	스타일 이름	특징
기본	default	matplotlib 기본 스타일
기본	classic	구버전 스타일
Seaborn	seaborn-v0_8	기본 seaborn 느낌
Seaborn	seaborn-v0_8-whitegrid	흰 배경 + 격자
Seaborn	seaborn-v0_8-darkgrid	어두운 격자
Seaborn	seaborn-v0_8-white	깔끔한 흰 배경
Seaborn	seaborn-v0_8-dark	어두운 배경
Seaborn	seaborn-v0_8-ticks	축 눈금 강조
Seaborn	seaborn-v0_8-paper	작은 글씨, 컴팩트
Seaborn	seaborn-v0_8-talk	글씨 큼
Seaborn	seaborn-v0_8-poster	매우 큼
기타	ggplot	R ggplot 스타일
기타	bmh	Bayesian 스타일
기타	grayscale	흑백
기타	dark_background	검정 배경
기타	fivethirtyeight	데이터 저널 스타일
기타	fast	빠른 렌더링

간단한 라인 플롯 (2)

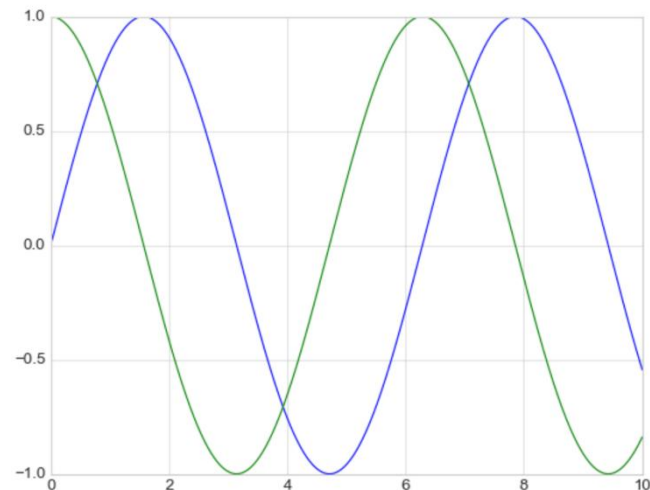
■ 라인 플롯 (2) - 예제들

```
fig = plt.figure()
ax = plt.axes()

x = np.linspace(0, 10, 1000)
ax.plot(x, np.sin(x));
plt.show()
```



```
plt.plot(x, np.sin(x))
plt.plot(x, np.cos(x));
plt.show()
```



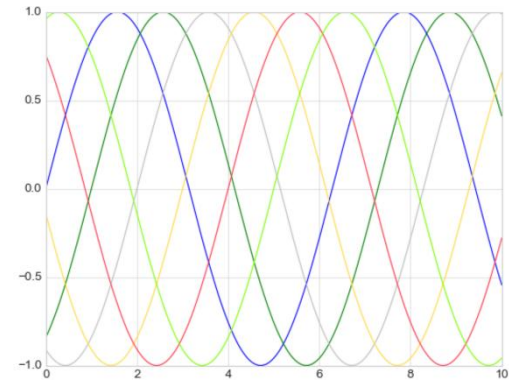
간단한 라인 플롯 (3)

■ 플롯 수정하기: 선 색상과 스타일

■ 색상 조정: **color** 키워드 사용

■ 색상을 지정하지 않으면 자동 채택

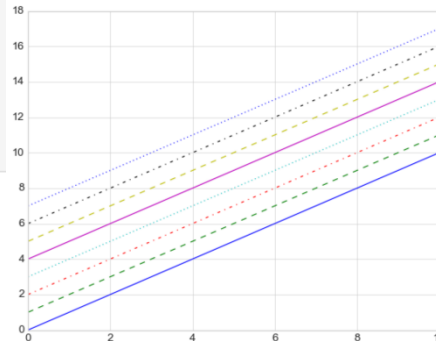
```
plt.plot(x, np.sin(x - 0), color='blue')      # specify color by name
plt.plot(x, np.sin(x - 1), color='g')        # short color code (rgbcmk)
plt.plot(x, np.sin(x - 2), color='0.75')     # Grayscale between 0 and 1
plt.plot(x, np.sin(x - 3), color='#FFDD44')  # Hex code (RRGGBB from 00 to FF)
plt.plot(x, np.sin(x - 4), color=(1.0,0.2,0.3)) # RGB tuple, values 0 to 1
plt.plot(x, np.sin(x - 5), color='chartreuse'); # all HTML color names supported
plt.show()
```



■ 선 스타일 조정: **linestyle** 키워드 사용

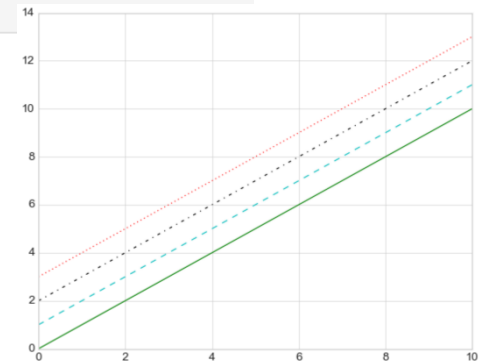
```
plt.plot(x, x + 0, linestyle='solid')
plt.plot(x, x + 1, linestyle='dashed')
plt.plot(x, x + 2, linestyle='dashdot')
plt.plot(x, x + 3, linestyle='dotted');

# For short, you can use the following codes:
plt.plot(x, x + 4, linestyle='-') # solid
plt.plot(x, x + 5, linestyle='--') # dashed
plt.plot(x, x + 6, linestyle='-.') # dashdot
plt.plot(x, x + 7, linestyle=':'); # dotted
plt.show()
```



■ 색상 조정과 스타일 단일 인수 사용

```
plt.plot(x, x + 0, '-g') # solid green
plt.plot(x, x + 1, '--c') # dashed cyan
plt.plot(x, x + 2, '-.k') # dashdot black
plt.plot(x, x + 3, ':r'); # dotted red
plt.show()
```



간단한 라인 플롯 (4)

■ 플롯 조정하기: 축 경계

■ 축 경계 조정

- `plt.xlim()`과 `plt.ylim()` 사용

- `plt.axis()` 사용: [x축 최솟값, x축 최댓값, y축 최솟값, y축 최댓값]

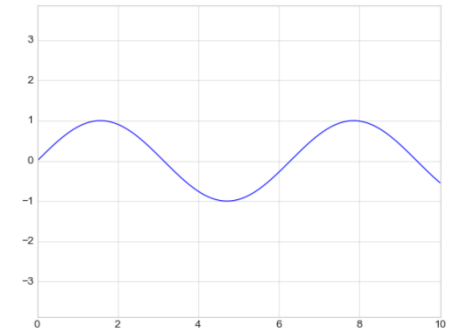
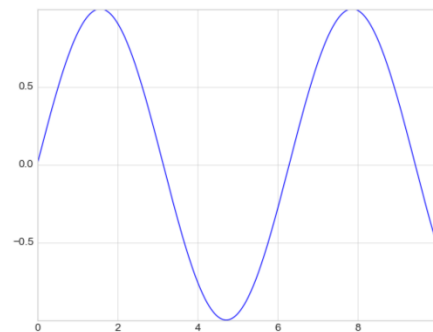
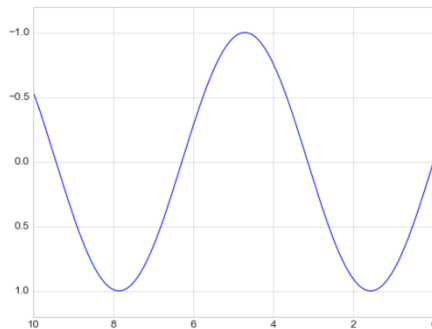
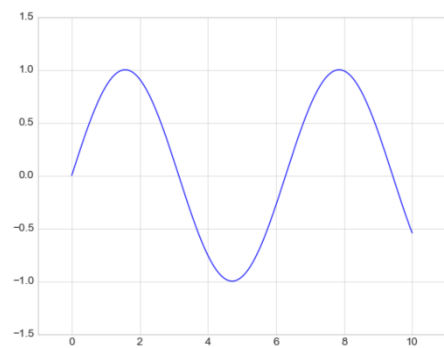
```
plt.plot(x, np.sin(x))  
plt.xlim(-1, 11)  
plt.ylim(-1.5, 1.5);  
plt.show()
```

```
plt.plot(x, np.sin(x))  
plt.axis([-1, 11, -1.5, 1.5]);  
plt.show()
```

```
plt.plot(x, np.sin(x))  
plt.xlim(10, 0)  
plt.ylim(1.2, -1.2);  
plt.show()
```

```
plt.plot(x, np.sin(x))  
plt.axis('tight');  
plt.show()
```

```
plt.plot(x, np.sin(x))  
plt.axis('equal');  
plt.show()
```



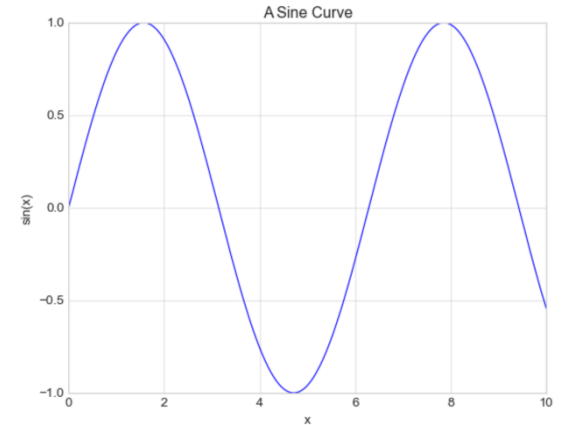
간단한 라인 플롯 (5)

■ 플롯 레이블 붙이기

■ 제목, 축, 간단한 범례 붙이기

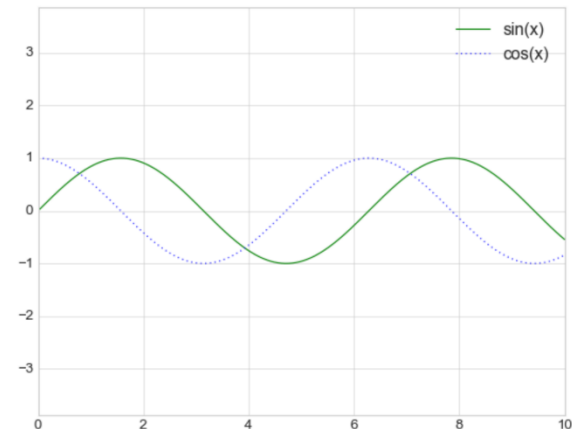
- title(): 제목
- xlabel(): x축 레이블
- ylabel(): y축 레이블
- legend(): 범례 생성

```
plt.plot(x, np.sin(x))  
plt.title("A Sine Curve")  
plt.xlabel("x")  
plt.ylabel("sin(x)");  
plt.show()
```



■ 라인 스타일과 색상을 기록, 레이블과 자동 매칭

```
plt.plot(x, np.sin(x), '-g', label='sin(x)')  
plt.plot(x, np.cos(x), ':b', label='cos(x)')  
plt.axis('equal')  
plt.legend();  
plt.show()
```



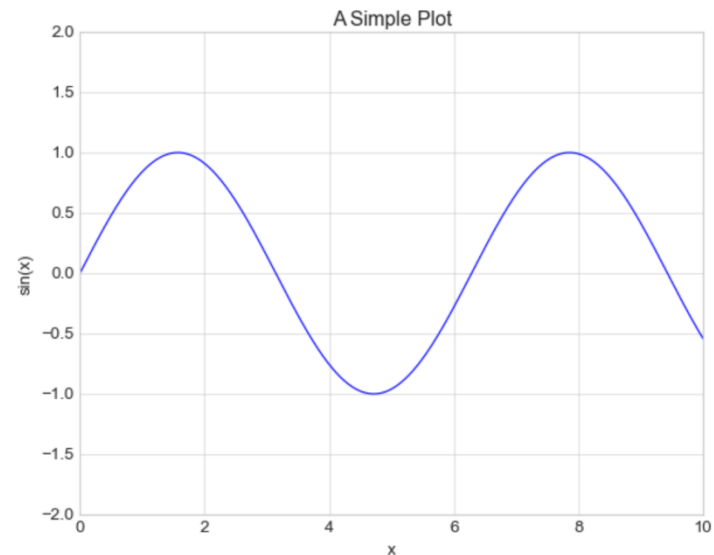
간단한 라인 플롯 (6)

▪ Matplotlib 주의사항

▪ 객체지향 인터페이스 메소드 사용

- `plt.plot()` → `ax.plot()`
- `plt.legend()` → `ax.legend()`
- `plt.xlabel()` → `ax.set_xlabel()`
- `plt.ylabel()` → `ax.set_ylabel()`
- `plt.xlim()` → `ax.set_xlim()`
- `plt.ylim()` → `ax.set_ylim()`
- `plt.title()` → `ax.set_title()`

```
ax = plt.axes()
ax.plot(x, np.sin(x))
ax.set(xlim=(0, 10), ylim=(-2, 2),
       xlabel='x', ylabel='sin(x)',
       title='A Simple Plot');
plt.show()
```



간단한 산점도

간단한 산점도 (1)

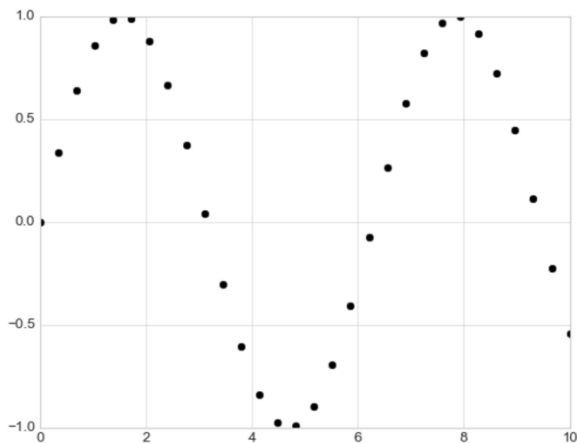
■ 산점도 (Scatter plot)

- 선 연결 대신 점(dot)이나 원, 또는 다른 모양으로 표현

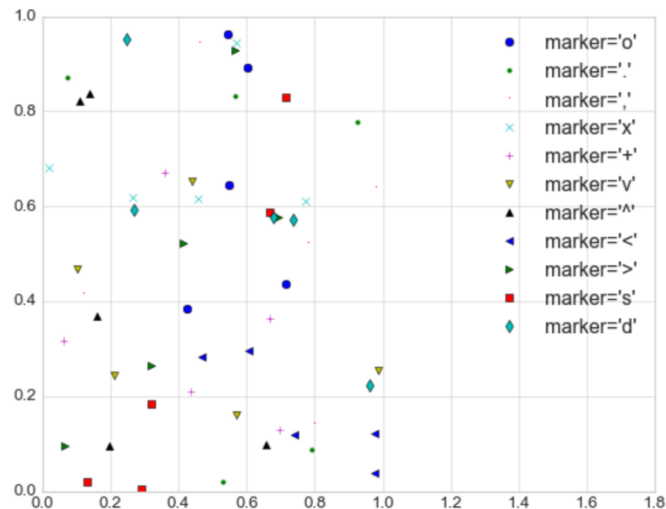
■ plt.plot()을 사용한 산점도

```
x = np.linspace(0, 10, 30)
y = np.sin(x)

plt.plot(x, y, 'o', color='black');
plt.show()
```



```
rng = np.random.RandomState(0)
for marker in ['o', '.', ',', 'x', '+', 'v', '^', '<', '>', 's', 'd']:
    plt.plot(rng.rand(5), rng.rand(5), marker,
             label="marker='{0}'".format(marker))
plt.legend(numpoints=1)
plt.xlim(0, 1.8);
plt.show()
```

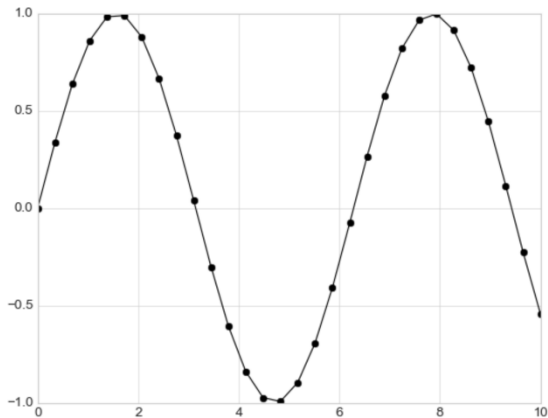


간단한 산점도 (2)

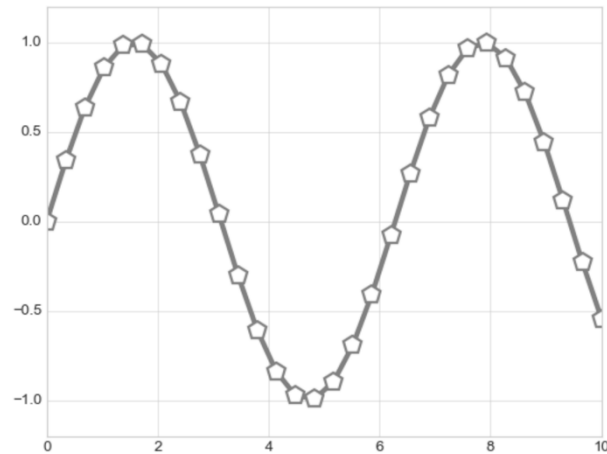
■ 산점도 (Scatter plot) (2)

- plt.plot()을 사용한 산점도 (cont')

```
plt.plot(x, y, '-ok');  
plt.show()
```



```
plt.plot(x, y, '-p', color='gray',  
         markersize=15, linewidth=4,  
         markerfacecolor='white',  
         markeredgecolor='gray',  
         markeredgewidth=2)  
plt.ylim(-1.2, 1.2);  
plt.show()
```

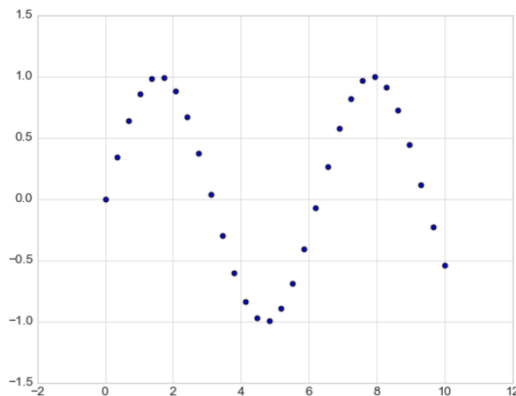


간단한 산점도 (3)

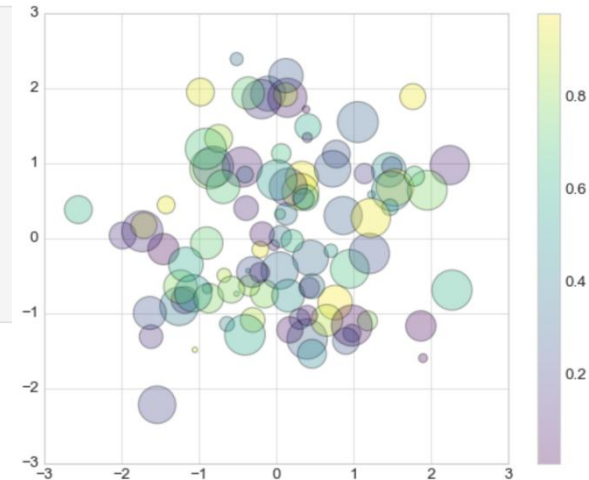
■ 산점도 (Scatter plot) (3)

- plt.scatter를 활용한 산점도
 - plt.plot 차이점은 plt.scatter의 경우 각 점의 속성 (크기, 표면 색상, 테두리 색상 등)을 개별적으로 제어하거나 데이터에 매핑 가능
 - 색상인수: 색상 척도에 자동으로 매핑 (colorbar())
 - 크기인수: 픽셀 단위로 주어짐

```
plt.scatter(x, y, marker='o');  
plt.show()
```



```
rng = np.random.RandomState(0)  
x = rng.randn(100)  
y = rng.randn(100)  
colors = rng.rand(100)  
sizes = 1000 * rng.rand(100)  
  
plt.scatter(x, y, c=colors, s=sizes, alpha=0.3,  
            cmap='viridis')  
plt.colorbar(); # show color scale  
plt.show()
```



간단한 산점도 (4)

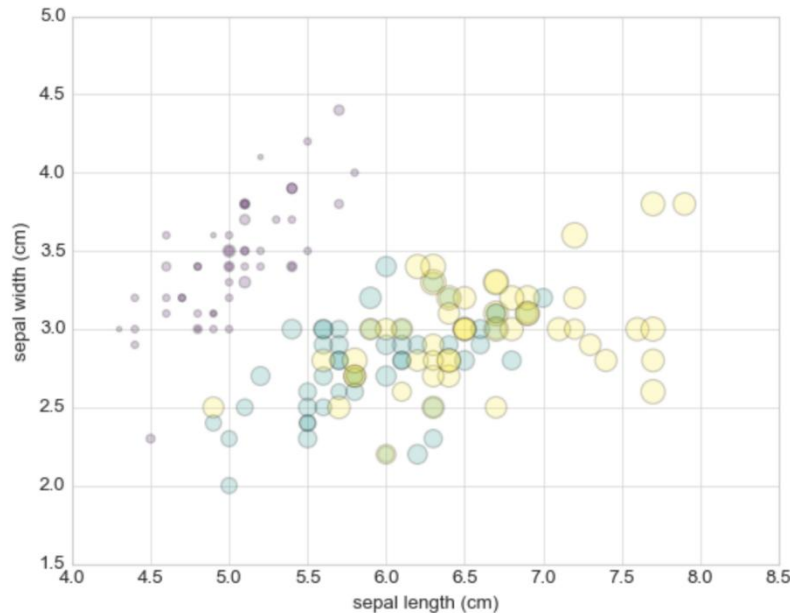
■ 산점도 (Scatter plot) (4)

- (예시) plt.scatter를 활용한 산점도 (cont')

```
from sklearn.datasets import load_iris
iris = load_iris()
features = iris.data.T
```

→ load_iris(): scikit-learn에서 제공하는 붓꽃 데이터

```
plt.scatter(features[0], features[1], alpha=0.2,
             s=100*features[3], c=iris.target, cmap='viridis')
plt.xlabel(iris.feature_names[0])
plt.ylabel(iris.feature_names[1]);
plt.show()
```



- 각 점 (x, y)는 꽃받침 길이와 폭,
- 점의 크기는 꽃잎의 폭,
- 색상은 꽃의 종

간단한 산점도 (5)

■ 오차 시각화하기 (1)

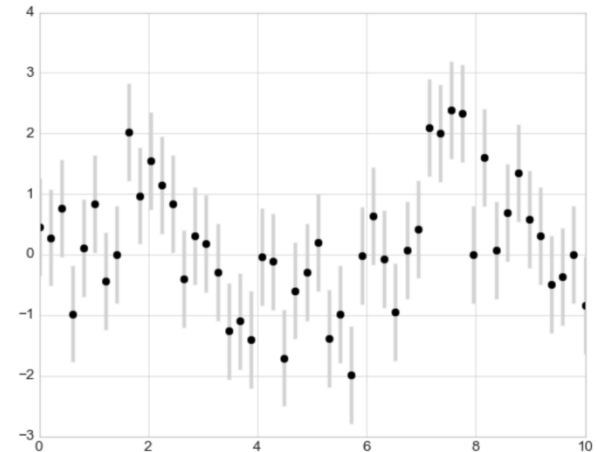
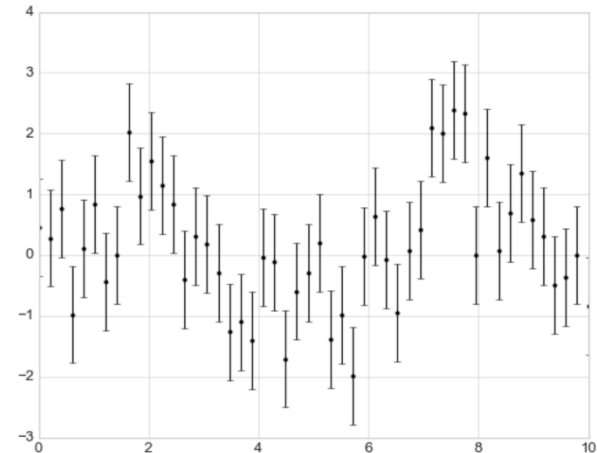
■ 기초 오차 막대

■ `errorbar()` 사용

■ `fmt`: 선과 점이 모양을 제어하는 포맷

```
x = np.linspace(0, 10, 50)
dy = 0.8
y = np.sin(x) + dy * np.random.randn(50)
plt.errorbar(x, y, yerr=dy, fmt='.k');
plt.show()
```

```
plt.errorbar(x, y, yerr=dy, fmt='o', color='black',
             ecol='lightgray', elinewidth=3, capsize=0);
plt.show()
```



밀로 플롯과 등고선 플롯

밀도 플롯과 등고선 플롯 (1)

■ 밀도 플롯과 등고선 플롯 (1)

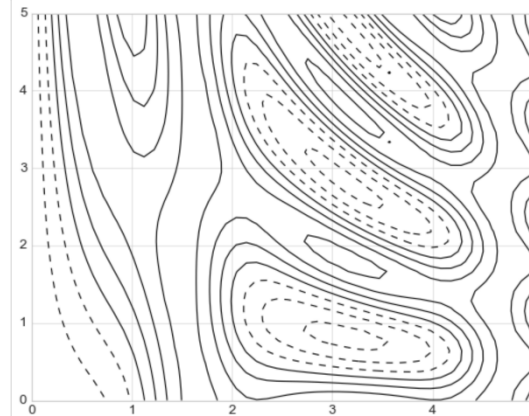
- `plt.contour()`: 등고선 플롯
- `plt.contourf()`: 색이 채워진 등고선 플롯
- (예시) 3차원 함수 시각화하기

```
def f(x, y):  
    return np.sin(x) ** 10 + np.cos(10 + y * x) * np.cos(x)
```

```
x = np.linspace(0, 5, 50)  
y = np.linspace(0, 5, 40)
```

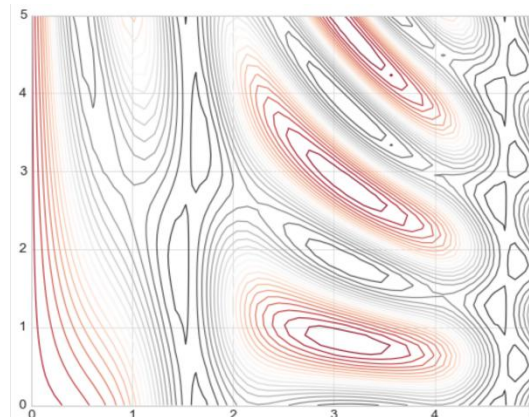
```
X, Y = np.meshgrid(x, y)  
Z = f(X, Y)
```

```
plt.contour(X, Y, Z, colors='black');  
plt.show()
```



```
# 데이터범위 내에 20개의 동일한 간격  
plt.contour(X, Y, Z, 20, cmap='RdGy');  
plt.show()
```

→ 'RdGy': Red-Gray 약어

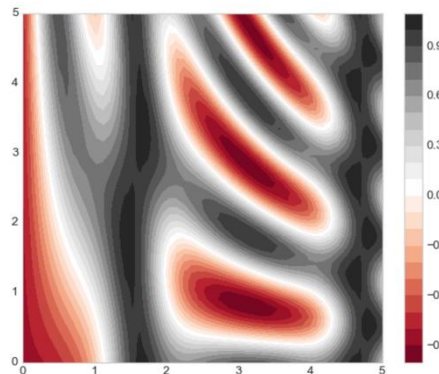


밀도 플롯과 등고선 플롯 (2)

■ 밀도 플롯과 등고선 플롯 (2)

- (예시) 색이 채워진 등고선 플롯

```
plt.contourf(X, Y, Z, 20, cmap='RdGy')  
plt.colorbar();  
plt.show()
```



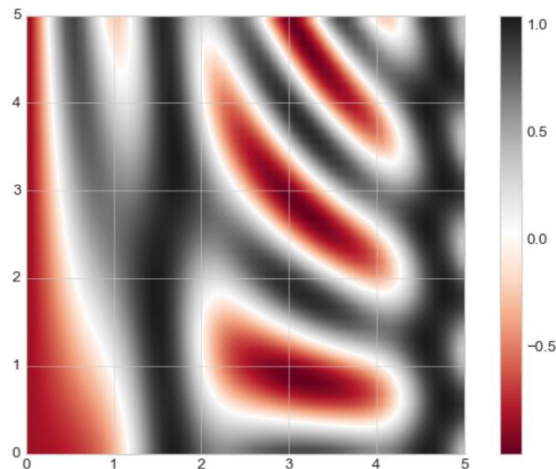
→ 등고선 레벨에 따라 렌더링
오래 걸리는 단점 !!

→ imshow() 사용 이미지 출력

- plt.imshow() 사용 주의사항

- x, y 그리드 입력을 받지 않으니 이미지의 extent[xmin, xmax, ymin, ymax]를 직접 지정
- 원점이 왼쪽 위에 위치 → 그리드 데이터 표현시 위치 변경 필수!!
- 입력 데이터에 매칭을 위해 자동으로 축의 가로세로 비율을 조정

```
plt.imshow(Z, extent=[0, 5, 0, 5], origin='lower',  
           cmap='RdGy')  
plt.colorbar()  
plt.axis('image');  
plt.show()
```

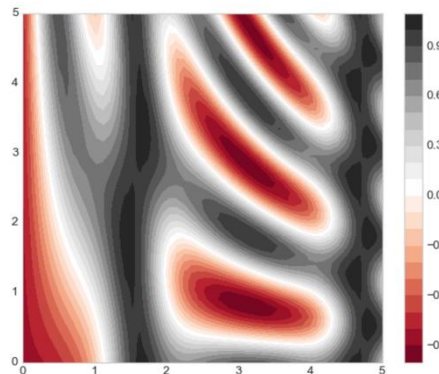


밀도 플롯과 등고선 플롯 (3)

■ 밀도 플롯과 등고선 플롯 (3)

- (예시) 색이 채워진 등고선 플롯

```
plt.contourf(X, Y, Z, 20, cmap='RdGy')  
plt.colorbar();  
plt.show()
```

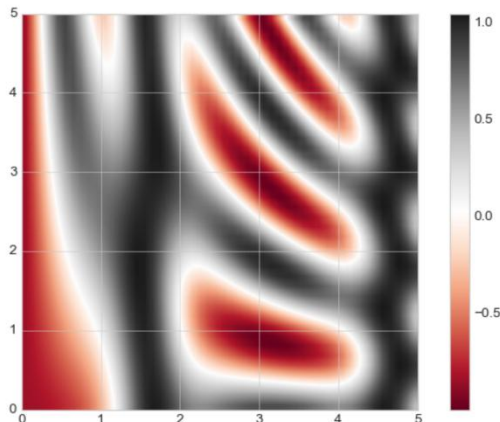


- ➔ 등고선 레벨에 따라 렌더링
오래 걸리는 단점 !!
- ➔ imshow() 사용 이미지 출력

- plt.imshow() 사용 주의사항

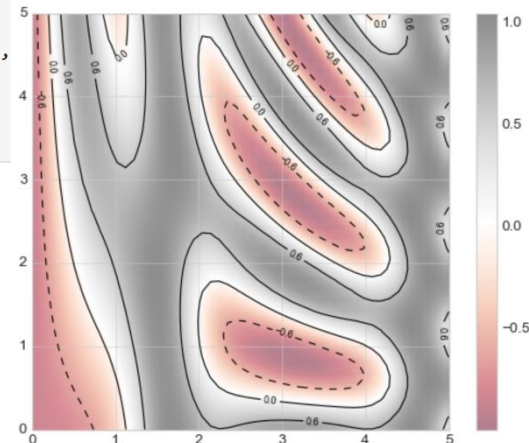
- x, y 그리드 입력을 받지 않으니 이미지의 extent[xmin, xmax, ymin, ymax]를 직접 지정
- 원점이 왼쪽 위에 위치 ➔ 그리드 데이터 표현시 위치 변경 필수!!
- 입력 데이터에 매칭을 위해 자동으로 축의 가로세로 비율을 조정

```
plt.imshow(Z, extent=[0, 5, 0, 5], origin='lower',  
           cmap='RdGy')  
plt.colorbar()  
plt.axis('image');  
plt.show()
```



```
contours = plt.contour(X, Y, Z, 3, colors='black')  
plt.clabel(contours, inline=True, fontsize=8)  
  
plt.imshow(Z, extent=[0, 5, 0, 5], origin='lower',  
           cmap='RdGy', alpha=0.5)  
plt.colorbar();  
plt.show()
```

- ➔ clabel(): 등고선 자체 라벨 추가
- ➔ alpha: 투명도



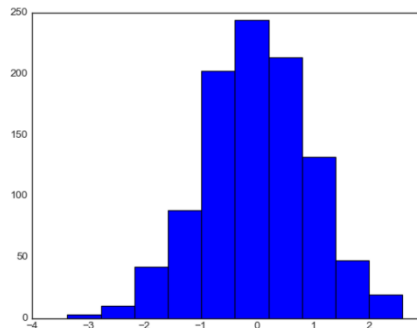
밀도 플롯과 등고선 플롯 (4)

■ 히스토그램, 구간화, 밀도

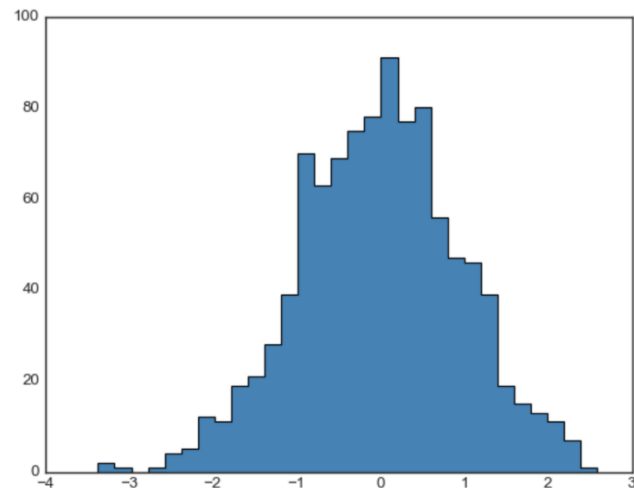
- hist(): 히스토그램 함수, 계산과 표현 모두 조정 가능

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
plt.style.use('seaborn-v0_8-white')

data = np.random.randn(1000)
plt.hist(data);
plt.show()
```



```
plt.hist(data, bins=30, density=True, alpha=0.5,
         histtype='stepfilled', color='steelblue',
         edgecolor='none');
plt.show()
```



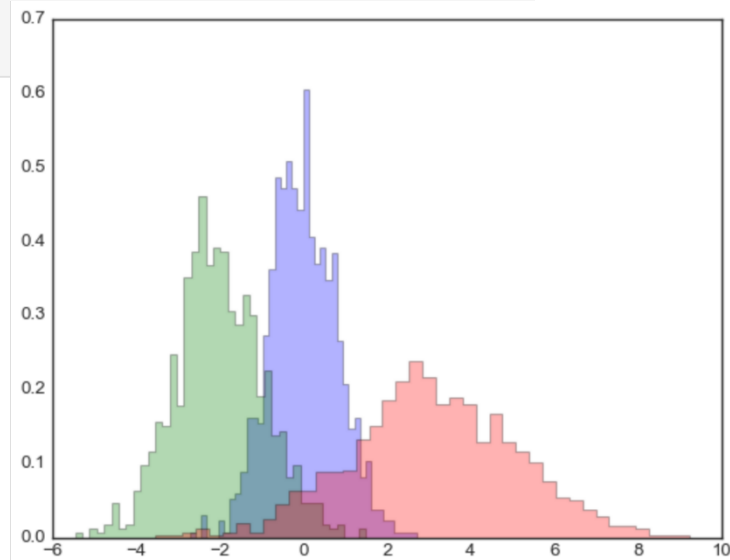
밀도 플롯과 등고선 플롯 (4)

- 히스토그램, 구간화, 밀도 (2)
 - (예시) 다양한 분포 히스토그램 비교

```
x1 = np.random.normal(0, 0.8, 1000)
x2 = np.random.normal(-2, 1, 1000)
x3 = np.random.normal(3, 2, 1000)

kwargs = dict(histtype='stepfilled', alpha=0.3, density=True, bins=40)

plt.hist(x1, **kwargs)
plt.hist(x2, **kwargs)
plt.hist(x3, **kwargs);
plt.show()
```



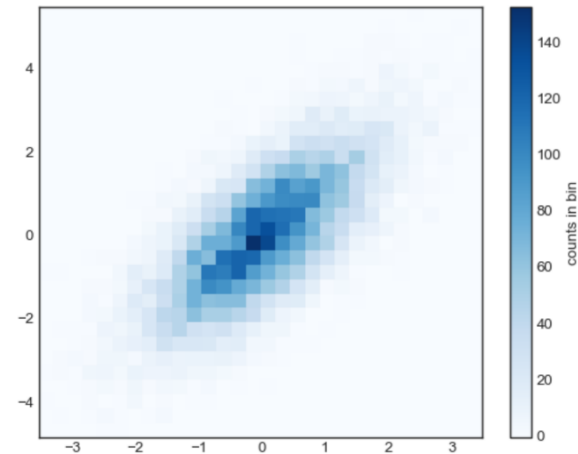
밀도 플롯과 등고선 플롯 (5)

■ 2차원 히스토그램과 구간화 (1)

- plt.hist2d: 2차원 히스토그램
- np.histogram2d: 2차원 히스토그램 계산

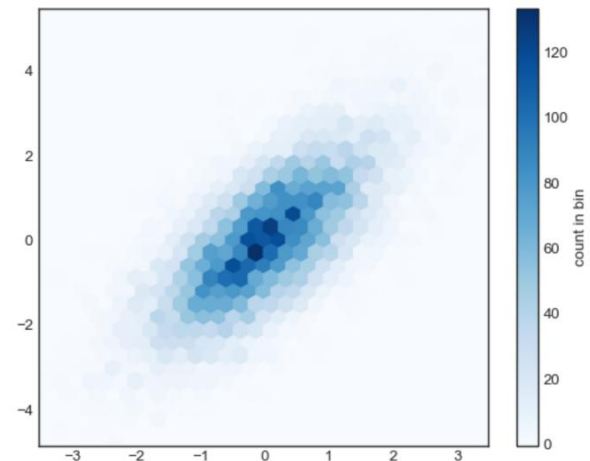
```
mean = [0, 0]
cov = [[1, 1], [1, 2]]
x, y = np.random.multivariate_normal(mean, cov, 10000).T
```

```
plt.hist2d(x, y, bins=30, cmap='Blues')
cb = plt.colorbar()
cb.set_label('counts in bin')
plt.show()
```



- plt.hexbin: 육각형 구간화

```
plt.hexbin(x, y, gridsize=30, cmap='Blues')
cb = plt.colorbar(label='count in bin')
plt.show()
```



밀도 플롯과 등고선 플롯 (6)

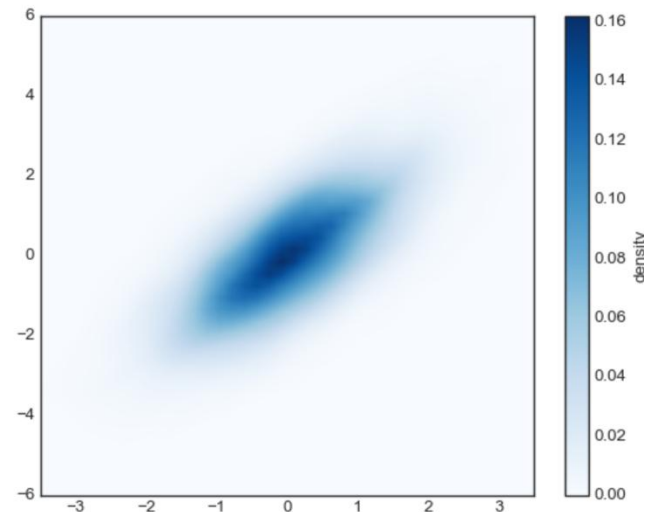
- 2차원 히스토그램과 구간화 (2)
 - 커널 밀도 추정 (KDE, Kernel density estimation)

```
from scipy.stats import gaussian_kde

# fit an array of size [Ndim, Nsamples]
data = np.vstack([x, y])
kde = gaussian_kde(data)

# evaluate on a regular grid
xgrid = np.linspace(-3.5, 3.5, 40)
ygrid = np.linspace(-6, 6, 40)
Xgrid, Ygrid = np.meshgrid(xgrid, ygrid)
Z = kde.evaluate(np.vstack([Xgrid.ravel(), Ygrid.ravel()]))

# Plot the result as an image
plt.imshow(Z.reshape(Xgrid.shape),
           origin='lower', aspect='auto',
           extent=[-3.5, 3.5, -6, 6],
           cmap='Blues')
cb = plt.colorbar()
cb.set_label("density")
plt.show()
```



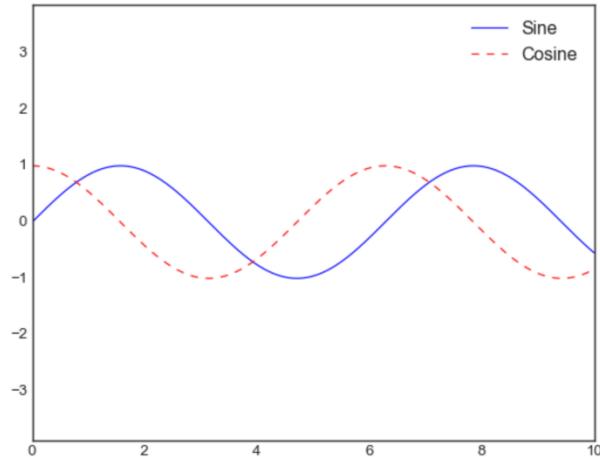
플롯 범례 맞춤 변경하기

플롯 범례 맞춤 변경하기 (1)

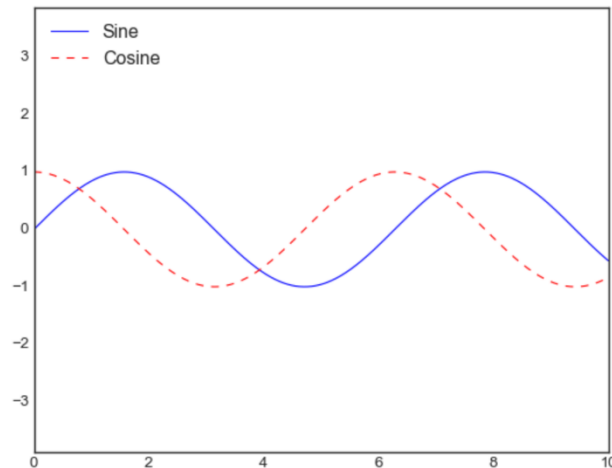
■ 플롯 범례

- 다양한 플롯 요소에 레이블을 할당
- `plt.legend()`

```
x = np.linspace(0, 10, 1000)
fig, ax = plt.subplots()
ax.plot(x, np.sin(x), '-b', label='Sine')
ax.plot(x, np.cos(x), '--r', label='Cosine')
ax.axis('equal')
leg = ax.legend();
plt.show()
```



```
ax.legend(loc='upper left', frameon=False)
fig
```

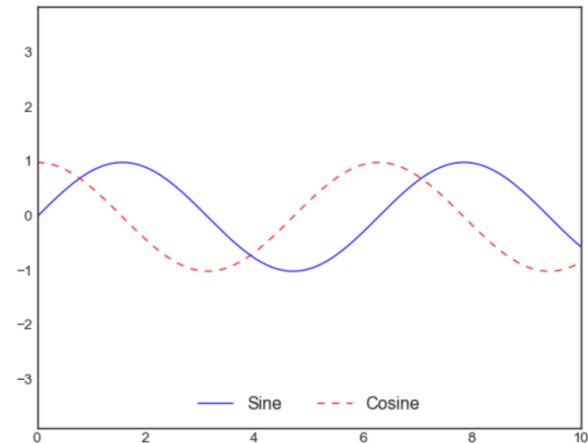


플롯 범례 맞춤 변경하기 (2)

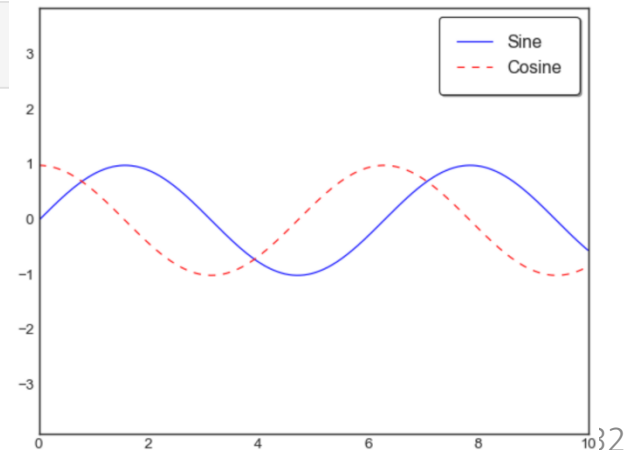
■ 플롯 범례 (2)

- `ncol()`: 범례의 열의 개수

```
ax.legend(frameon=False, loc='lower center', ncol=2)  
fig
```



```
ax.legend(frameon=True, fancybox=True, framealpha=1, shadow=True, borderpad=1)  
fig
```

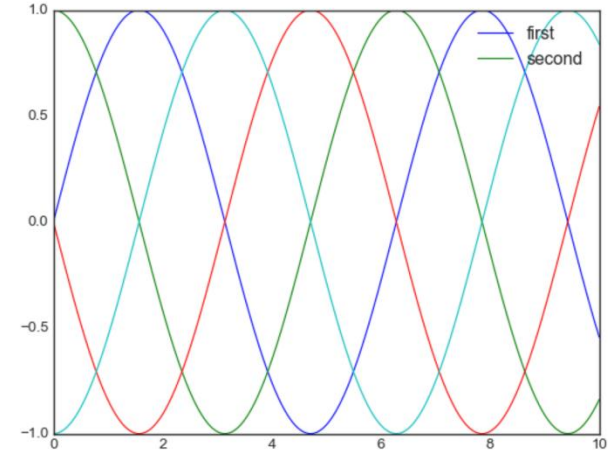


플롯 범례 맞춤 변경하기 (3)

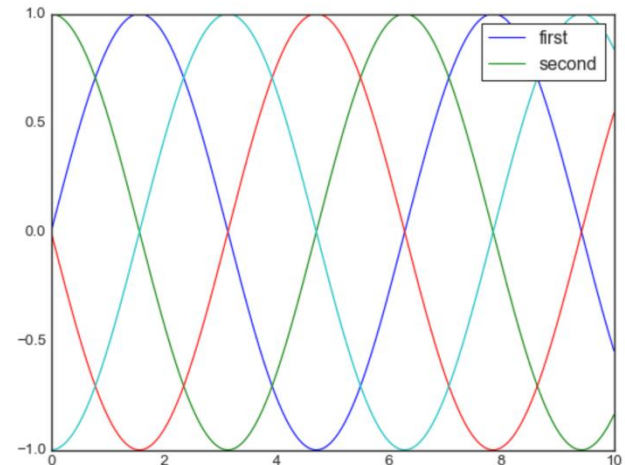
■ 범례에 사용할 요소 선택하기

- plot 명령어가 반환하는 객체를 사용에 범례 조정
- plt.plot() 명령어는 라인 인스턴스의 목록을 반환

```
y = np.sin(x[:, np.newaxis] + np.pi * np.arange(0, 2, 0.5))  
lines = plt.plot(x, y)  
  
# lines is a list of plt.Line2D instances  
plt.legend(lines[:2], ['first', 'second']);  
plt.show()
```



```
plt.plot(x, y[:, 0], label='first')  
plt.plot(x, y[:, 1], label='second')  
plt.plot(x, y[:, 2:])  
plt.legend(framealpha=1, frameon=True);  
plt.show()
```



플롯 범례 맞춤 변경하기 (4)

■ 점 크기에 대한 범례

- 원래 범례 표시를 하려면 플롯상의 객체를 참조하기에 항상 플로팅해야 함
- 원하는 객체가 플롯상에 없다면 → 빈 리스트를 플로팅해 가짜로 객체 생성하여 표시

```
import pandas as pd
cities = pd.read_csv('c:/Source_lecture/california_cities.csv')

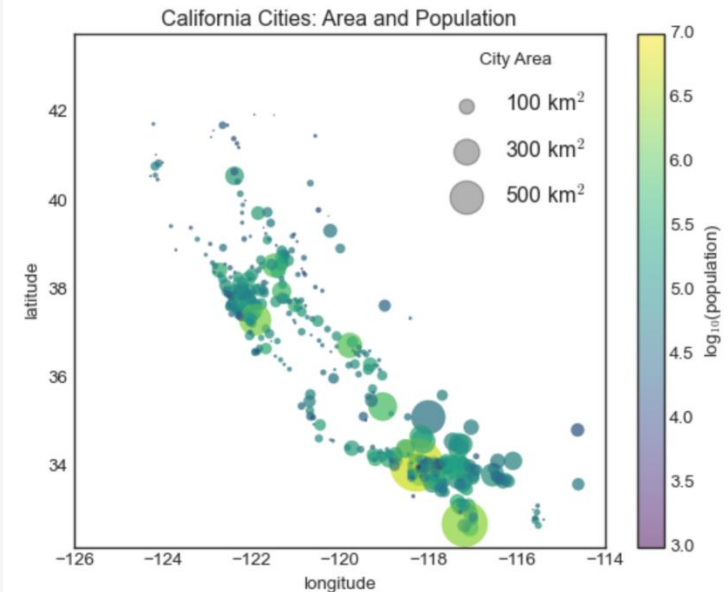
# Extract the data we're interested in
lat, lon = cities['latd'], cities['longd']
population, area = cities['population_total'], cities['area_total_km2']

# Scatter the points, using size and color but no label
plt.scatter(lon, lat, label=None,
            c=np.log10(population), cmap='viridis',
            s=area, linewidth=0, alpha=0.5)

plt.axis('equal')
plt.xlabel('longitude')
plt.ylabel('latitude')
plt.colorbar(label='log10$(population)$')
plt.clim(3, 7)

# Here we create a legend:
# we'll plot empty lists with the desired size and label
for area in [100, 300, 500]:
    plt.scatter([], [], c='k', alpha=0.3, s=area,
                label=str(area) + ' km$^2$')
plt.legend(scatterpoints=1, frameon=False, labelspacing=1, title='City Area')

plt.title('California Cities: Area and Population');
plt.show()
```



플롯 범례 맞춤 변경하기 (5)

■ 다중 범례

- 표준 legend() 인터페이스는 하나의 범례만 생성 가능
- 새로운 범례 아티스트를 만들고, 하위 수준의 ax.add_artist() 메소드를 사용해 두 번째 아티스트를 플롯에 직접 추가하는 방식 사용

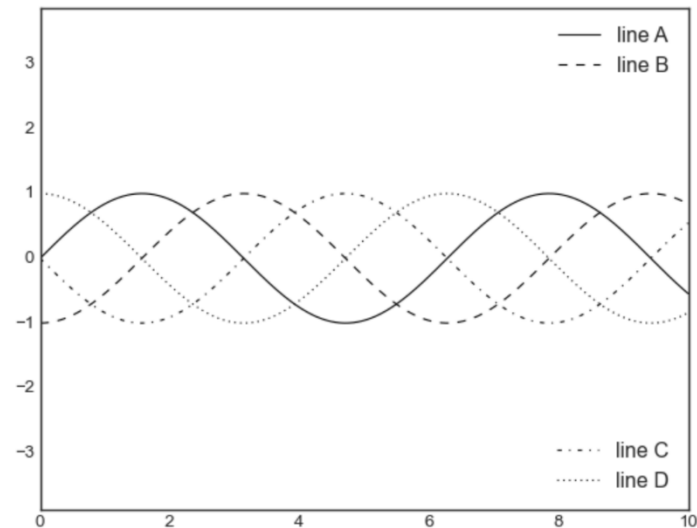
```
fig, ax = plt.subplots()

lines = []
styles = ['-', '--', '-.', ':']
x = np.linspace(0, 10, 1000)

for i in range(4):
    lines += ax.plot(x, np.sin(x - i * np.pi / 2),
                    styles[i], color='black')
ax.axis('equal')

# specify the lines and labels of the first legend
ax.legend(lines[:2], ['line A', 'line B'],
        loc='upper right', frameon=False)

# Create the second legend and add the artist manually.
from matplotlib.legend import Legend
leg = Legend(ax, lines[2:], ['line C', 'line D'],
            loc='lower right', frameon=False)
ax.add_artist(leg);
plt.show()
```



색상 막대 맞춤 변경하기

색상 막대 맞춤 변경하기 (1)

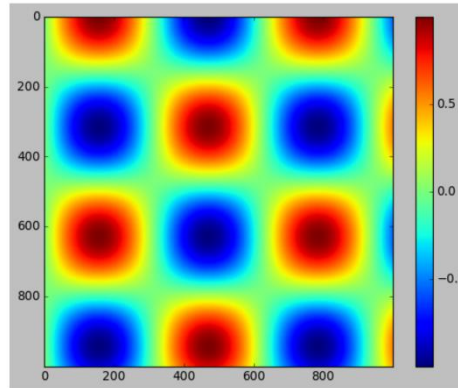
■ 색상 막대

- 점, 선 및 영역의 색상을 기반으로 하는 연속적인 레이블일 경우 사용

```
import matplotlib.pyplot as plt
plt.style.use('classic')

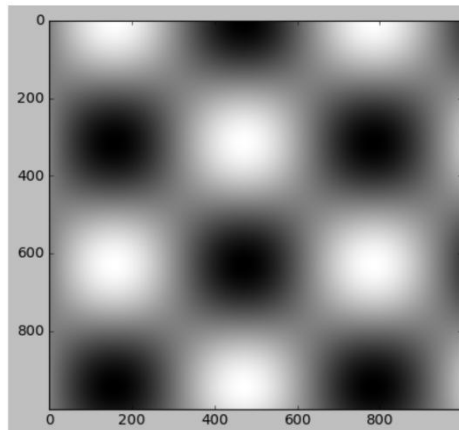
x = np.linspace(0, 10, 1000)
I = np.sin(x) * np.cos(x[:, np.newaxis])

plt.imshow(I)
plt.colorbar();
plt.show()
```



■ 색상 막대 맞춤 변경하기 (1)

```
plt.imshow(I, cmap='gray')
plt.show()
```

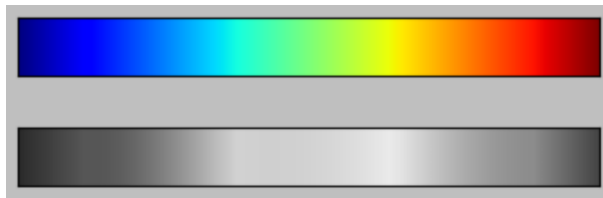


색상 막대 맞춤 변경하기 (2)

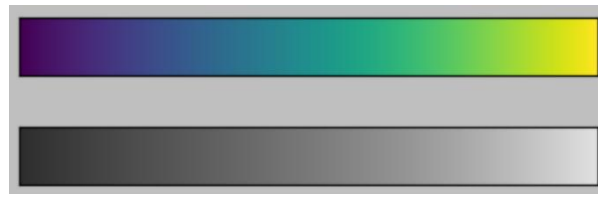
■ 색상 막대 (2)

■ 색상 지도 선택하기

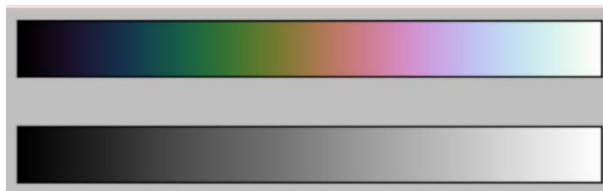
- 순차적 색상 지도: 하나의 연속적인 순서를 가지는 색상으로 구성 (예, binary 또는 viridis)
- 발산적 색상 지도: 평균을 기준으로 양수 및 음수 편차를 나타내는 두 가지 구별된 색상 포함 (예, RdBu 또는 PuOr)
- 정성적 색상 지도: 특성 순서 없이 색을 혼합 (예, rainbow 또는 jet)



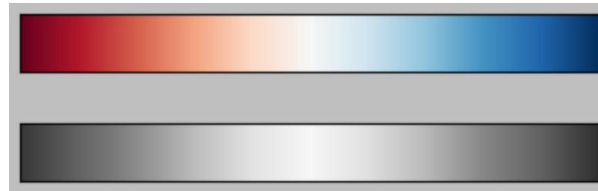
jet 색상지도와 휘도



viridis 색상지도와 휘도



cubehelix 색상지도와 휘도



RdBu 색상지도와 휘도

색상 막대 맞춤 변경하기 (3)

■ 색상 막대 (3)

■ 색상 한계 및 확장

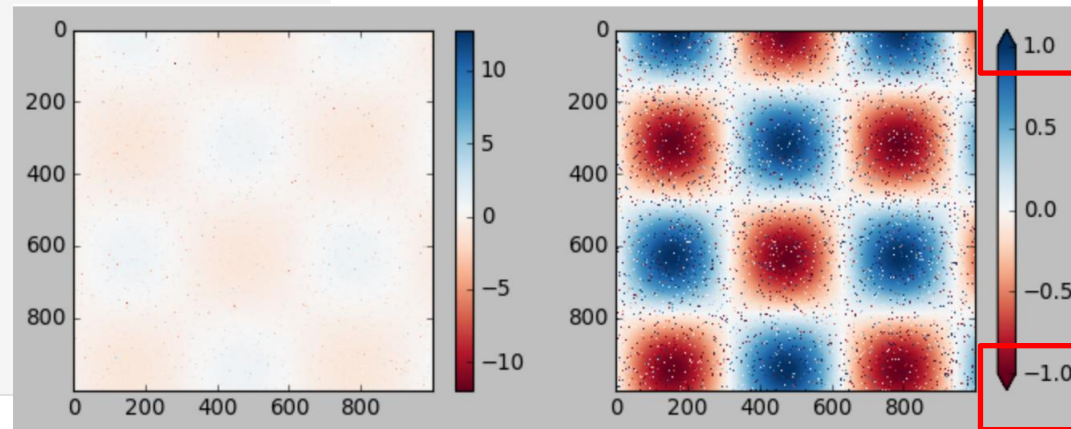
```
# make noise in 1% of the image pixels
speckles = (np.random.random(I.shape) < 0.01)
I[speckles] = np.random.normal(0, 3, np.count_nonzero(speckles))

plt.figure(figsize=(10, 3.5))

plt.subplot(1, 2, 1)
plt.imshow(I, cmap='RdBu')
plt.colorbar()

plt.subplot(1, 2, 2)
plt.imshow(I, cmap='RdBu')
plt.colorbar(extend='both')
plt.clim(-1, 1);
plt.show()
```

→ extend 속성을 설정해서 위아래에 삼각형 화살표로 색 한계를 좁히고 범위를 벗어난 값을 표시 가능



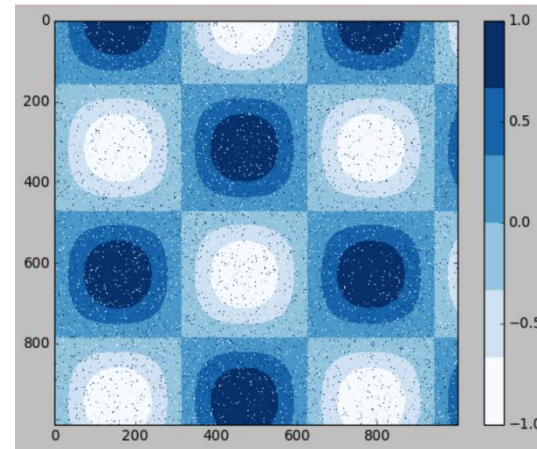
색상 막대 맞춤 변경하기 (4)

- 색상 막대 (4)

- 불연속적인 값을 표현할 경우

- `plt.cm.get_cmap()`: 적합한 색상 지도의 이름과 함께 원하는 구간(bin) 개수 전달

```
plt.imshow(I, cmap=plt.cm.get_cmap('Blues', 6))  
plt.colorbar()  
plt.clim(-1, 1);  
plt.show()
```



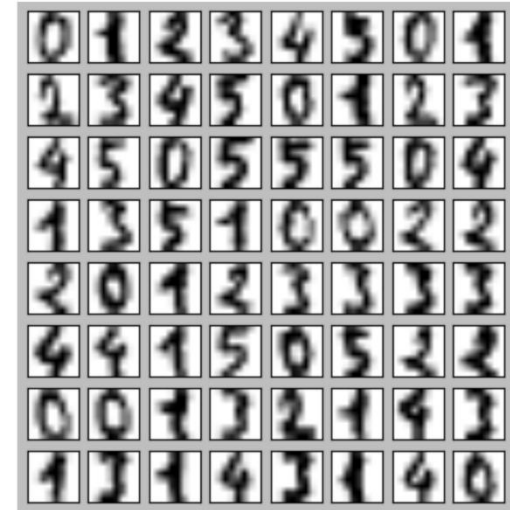
색상 막대 맞춤 변경하기 (5)

▪ (예시) 손으로 쓴 숫자

- Scikit-Learn에 포함
- 손으로 쓴 숫자 2000여개의 8 x 8 픽셀로 구성

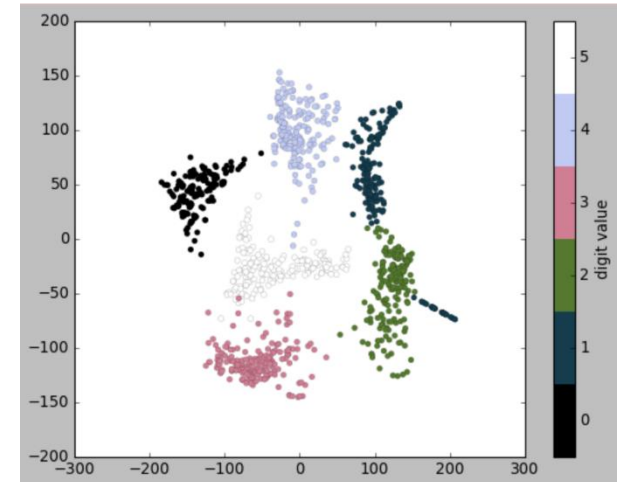
```
# Load images of the digits 0 through 5 and visualize several of them
from sklearn.datasets import load_digits
digits = load_digits(n_class=6)

fig, ax = plt.subplots(8, 8, figsize=(6, 6))
for i, axi in enumerate(ax.flat):
    axi.imshow(digits.images[i], cmap='binary')
    axi.set(xticks=[], yticks=[])
plt.show()
```



```
# project the digits into 2 dimensions using IsoMap
from sklearn.manifold import Isomap
iso = Isomap(n_components=2)
projection = iso.fit_transform(digits.data)
```

```
# plot the results
plt.scatter(projection[:, 0], projection[:, 1], lw=0.1,
            c=digits.target, cmap=plt.cm.get_cmap('cubehelix', 6))
plt.colorbar(ticks=range(6), label='digit value')
plt.clim(-0.5, 5.5)
plt.show()
```



Q & A
